FastAPI Interview Questions for AI/ML Roles

This README provides 170 interview questions tailored for AI/ML roles, focusing on **FastAPI**, a modern, high-performance web framework for building APIs in Python. The questions cover **core concepts** (e.g., routing, middleware, Pydantic models, database integration, ML model serving, testing, deployment, and scalability) and their applications in AI/ML workflows, such as serving machine learning models, integrating with LLMs, or handling data pipelines. Questions are categorized by topic and divided into **Basic**, **Intermediate**, and **Advanced** levels to support candidates preparing for roles requiring expertise in building and deploying APIs for AI-driven applications.

FastAPI Basics

Basic

1. **What is FastAPI, and why is it used in AI/ML applications?**
   A Python framework for building fast, asynchronous APIs, ideal for serving ML models.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def root():
    return {"message": "Hello, FastAPI!"}
```

2. **How do you install FastAPI and its dependencies?**
   Uses pip to install FastAPI and Uvicorn.

```python
# Terminal command
pip install fastapi uvicorn
```

3. **What is the role of Pydantic in FastAPI?**
   Validates and serializes data for API requests/responses.

```python
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    price: float
```

4. **How do you run a FastAPI application?**
   Uses Uvicorn as the ASGI server.

```python
# Terminal command
uvicorn main:app --reload
```

5. **How do you define a simple GET endpoint in FastAPI?**
   Uses decorators for routing.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}
```

6. **How do you visualize API response times?**
   Plots latency metrics.

```python
import matplotlib.pyplot as plt
def plot_response_times(times):
    plt.plot(times)
    plt.savefig("response_times.png")
```

Intermediate

7. **Write a function to create a POST endpoint with Pydantic validation.**
   Accepts and validates JSON data.

```python
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
class Item(BaseModel):
    name: str
    price: float
@app.post("/items/")
async def create_item(item: Item):
    return item
```

8. **How do you handle query parameters in FastAPI?**
   Defines optional parameters in endpoints.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/items/")
```

```python
async def read_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

9. **Write a function to return custom responses in FastAPI.**
   Uses Response objects for status codes.

```python
from fastapi import FastAPI, Response
app = FastAPI()
@app.get("/health")
async def health_check():
    return Response(content="OK", status_code=200)
```

10. **How do you implement path parameters with type hints?**
    Enforces data types for parameters.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/users/{user_id}")
async def read_user(user_id: str):
    return {"user_id": user_id}
```

11. **Write a function to handle file uploads in FastAPI.**
    Processes uploaded files.

```python
from fastapi import FastAPI, File, UploadFile
app = FastAPI()
@app.post("/upload/")
async def upload_file(file: UploadFile = File(...)):
    return {"filename": file.filename}
```

12. **How do you enable automatic Swagger UI in FastAPI?**
    Included by default at `/docs` .

```python
from fastapi import FastAPI
app = FastAPI(docs_url="/docs")
@app.get("/")
async def root():
    return {"message": "Visit /docs for Swagger UI"}
```

Advanced

13. **Write a function to implement custom exception handling.**
    Returns custom error responses.

```python
from fastapi import FastAPI, HTTPException
app = FastAPI()
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id < 0:
        raise HTTPException(status_code=400, detail="Invalid item ID")
    return {"item_id": item_id}
```

14. **How do you implement dependency injection in FastAPI?**
    Reuses logic across endpoints.

```python
from fastapi import FastAPI, Depends
app = FastAPI()
async def get_db():
    return {"db": "connected"}
@app.get("/data")
async def read_data(db=Depends(get_db)):
    return db
```

15. **Write a function to implement background tasks in FastAPI.**
    Runs tasks asynchronously.

```python
from fastapi import FastAPI, BackgroundTasks
app = FastAPI()
async def log_task(message: str):
    print(f"Logging: {message}")
@app.post("/log/")
async def create_log(message: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(log_task, message)
    return {"status": "Task queued"}
```

16. **How do you implement WebSockets in FastAPI?**
    Handles real-time communication.

```python
from fastapi import FastAPI, WebSocket
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    await websocket.send_text("Connected")
    await websocket.close()
```

17. **Write a function to implement middleware in FastAPI.**
    Processes requests globally.

```python
from fastapi import FastAPI
app = FastAPI()
@app.middleware("http")
async def add_header(request, call_next):
    response = await call_next(request)
    response.headers["X-Custom-Header"] = "Value"
    return response
```

18. **How do you optimize FastAPI for high-concurrency?**
    Uses async/await and Uvicorn workers.

```python
# Terminal command
uvicorn main:app --workers 4 --host 0.0.0.0 --port 8000
```

Routing and Endpoints

Basic

19. **What is routing in FastAPI?**
    Maps URLs to functions.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/hello")
async def hello():
    return {"message": "Hello, World!"}
```

20. **How do you define multiple HTTP methods for an endpoint?**
    Uses separate decorators.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}
@app.post("/items/{item_id}")
async def update_item(item_id: int):
    return {"updated_id": item_id}
```

21. **What are path parameters in FastAPI?**
    Variables in URL paths.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/products/{product_id}")
async def read_product(product_id: int):
    return {"product_id": product_id}
```

22. **How do you handle optional query parameters?**
    Uses default values.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/search")
async def search_items(q: str | None = None):
    return {"query": q}
```

23. **How do you define a root endpoint in FastAPI?**
    Maps to `/`.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def root():
    return {"message": "Welcome to FastAPI"}
```

24. **How do you visualize endpoint usage?**
    Plots request counts.

```python
import matplotlib.pyplot as plt
def plot_endpoint_usage(counts):
    plt.bar(counts.keys(), counts.values())
    plt.savefig("endpoint_usage.png")
```

Intermediate

25. **Write a function to create nested routes.**
    Organizes endpoints hierarchically.

```python
from fastapi import FastAPI, APIRouter
app = FastAPI()
router = APIRouter(prefix="/api")
@router.get("/users")
async def read_users():
```

```python
    return {"users": []}
app.include_router(router)
```

26. **How do you implement dynamic routing?**
    Uses path parameters for flexibility.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/{resource}/{id}")
async def read_resource(resource: str, id: int):
    return {"resource": resource, "id": id}
```

27. **Write a function to handle route-specific middleware.**
    Applies logic to specific routes.

```python
from fastapi import FastAPI, Request
app = FastAPI()
@app.middleware("http")
async def log_specific_route(request: Request, call_next):
    if request.url.path.startswith("/admin"):
        print("Admin route accessed")
    return await call_next(request)
```

28. **How do you implement route versioning?**
    Uses prefixes for API versions.

```python
from fastapi import FastAPI, APIRouter
app = FastAPI()
v1 = APIRouter(prefix="/v1")
@v1.get("/items")
async def read_items_v1():
    return {"version": "v1"}
app.include_router(v1)
```

29. **Write a function to create tagged routes.**
    Groups endpoints in Swagger UI.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/items/", tags=["Items"])
async def read_items():
    return {"items": []}
```

30. **How do you handle route conflicts in FastAPI?**
    Prioritizes specific routes.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/items/{item_id}")
async def read_specific_item(item_id: int):
    return {"item_id": item_id}
@app.get("/items/all")
async def read_all_items():
    return {"items": "all"}
```

Advanced

31. **Write a function to implement dynamic route generation.**
    Creates routes programmatically.

```python
from fastapi import FastAPI
app = FastAPI()
def add_dynamic_route(path: str, response: dict):
    @app.get(path)
    async def dynamic_route():
        return response
add_dynamic_route("/custom", {"message": "Dynamic route"})
```

32. **How do you implement rate limiting for routes?**
    Uses middleware for throttling.

```python
from fastapi import FastAPI
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
app = FastAPI()
limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(429, _rate_limit_exceeded_handler)
@app.get("/limited")
@limiter.limit("5/minute")
async def limited_endpoint():
    return {"status": "OK"}
```

33. **Write a function to implement route-specific dependencies.**
    Applies custom logic to routes.

```python
from fastapi import FastAPI, Depends
app = FastAPI()
async def check_role():
    return {"role": "admin"}
@app.get("/admin", dependencies=[Depends(check_role)])
async def admin_endpoint():
    return {"access": "granted"}
```

34. **How do you optimize route performance?**
    Minimizes middleware and dependencies.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/fast")
async def fast_endpoint():
    return {"status": "optimized"}
```

35. **Write a function to monitor route performance.**
    Logs request latency.

```python
import time
from fastapi import FastAPI, Request
app = FastAPI()
@app.middleware("http")
async def log_performance(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    print(f"Route {request.url.path} took {time.time() - start}s")
    return response
```

36. **How do you implement route failover?**
    Retries failed routes.

```python
from fastapi import FastAPI
from tenacity import retry, stop_after_attempt
app = FastAPI()
@app.get("/reliable")
@retry(stop=stop_after_attempt(3))
async def reliable_endpoint():
    return {"status": "OK"}
```

Database Integration

Basic

37. **How do you integrate FastAPI with SQLAlchemy?**
    Uses ORM for database operations.

```python
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
Base = declarative_base()
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
engine = create_engine("sqlite:///example.db")
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
```

38. **How do you perform CRUD operations with FastAPI?**
    Implements create, read, update, delete.

```python
from fastapi import FastAPI
app = FastAPI()
@app.post("/users/")
async def create_user(name: str):
    session = Session()
    user = User(name=name)
    session.add(user)
    session.commit()
    return {"name": name}
```

39. **What is dependency injection for database sessions?**
    Provides sessions to endpoints.

```python
from fastapi import FastAPI, Depends
app = FastAPI()
def get_db():
    session = Session()
    try:
        yield session
    finally:
        session.close()
@app.get("/users")
async def read_users(db=Depends(get_db)):
    return db.query(User).all()
```

40. **How do you connect FastAPI to PostgreSQL?**
    Uses SQLAlchemy with PostgreSQL URI.

```python
engine = create_engine("postgresql://user:password@localhost:5432/dbname")
```

41. **How do you handle database migrations in FastAPI?**
    Uses Alembic for schema changes.

```python
# Terminal command
alembic init migrations
```

42. **How do you visualize database query performance?**
    Plots query execution times.

```python
import matplotlib.pyplot as plt
def plot_query_times(times):
    plt.plot(times)
    plt.savefig("query_times.png")
```

Intermediate

43. **Write a function to implement async database operations.**
    Uses async SQLAlchemy.

```python
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from fastapi import FastAPI
app = FastAPI()
engine = create_async_engine("sqlite+aiosqlite:///example.db")
async def get_async_db():
    async with AsyncSession(engine) as session:
        yield session
@app.get("/users")
async def read_users(db=Depends(get_async_db)):
    return await db.execute("SELECT * FROM users").fetchall()
```

44. **How do you implement database connection pooling?**
    Configures SQLAlchemy for pooling.

```python
engine = create_engine("sqlite:///example.db", pool_size=5,
max_overflow=10)
```

45. **Write a function to handle database transactions.**
    Ensures atomic operations.

```python
from fastapi import FastAPI, Depends
app = FastAPI()
@app.post("/users")
async def create_user(name: str, db=Depends(get_db)):
    try:
        user = User(name=name)
        db.add(user)
        db.commit()
    except:
        db.rollback()
        raise
    return {"name": name}
```

46. **How do you integrate FastAPI with MongoDB?**
    Uses Motor for async MongoDB access.

```python
from motor.motor_asyncio import AsyncIOMotorClient
from fastapi import FastAPI
app = FastAPI()
client = AsyncIOMotorClient("mongodb://localhost:27017")
db = client["mydb"]
@app.post("/docs")
async def create_doc(data: dict):
    result = await db["collection"].insert_one(data)
    return {"id": str(result.inserted_id)}
```

47. **Write a function to cache database queries.**
    Reduces database load.

```python
from functools import lru_cache
@lru_cache(maxsize=100)
def cached_query(query: str):
    session = Session()
    result = session.execute(query).fetchall()
    session.close()
    return result
```

48. **How do you handle database connection errors?**
    Implements retries.

```python
from tenacity import retry, stop_after_attempt
@retry(stop=stop_after_attempt(3))
def connect_db():
    return create_engine("sqlite:///example.db")
```

Advanced

49. **Write a function to implement sharding in FastAPI.**
    Distributes data across databases.

```python
from fastapi import FastAPI
app = FastAPI()
engines = {i: create_engine(f"sqlite:///shard_{i}.db") for i in range(3)}
def get_shard_db(shard_key: int):
    return Session(bind=engines[shard_key % 3])
@app.get("/users/{user_id}")
async def read_user(user_id: int, db=Depends(get_shard_db)):
    return db.query(User).filter(User.id == user_id).first()
```

50. **How do you optimize database queries in FastAPI?**
    Uses indexing and query planning.

```python
from sqlalchemy import Index
Index("idx_user_name", User.name).create(bind=engine)
```

51. **Write a function to implement database failover.**
    Switches to backup database.

```python
from fastapi import FastAPI
app = FastAPI()
primary_engine = create_engine("sqlite:///primary.db")
backup_engine = create_engine("sqlite:///backup.db")
def get_db():
    try:
        return Session(bind=primary_engine)
    except:
        return Session(bind=backup_engine)
```

52. **How do you integrate FastAPI with Redis?**
    Uses Redis for caching.

```python
import redis.asyncio as redis
from fastapi import FastAPI
app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
@app.get("/cache/{key}")
async def read_cache(key: str):
    value = await r.get(key)
    return {"value": value.decode() if value else None}
```

53. **Write a function to implement database migrations programmatically.**
    Automates schema updates.

```python
from alembic.config import Config
from alembic import command
def run_migrations():
    alembic_cfg = Config("alembic.ini")
    command.upgrade(alembic_cfg, "head")
```

54. **How do you monitor database performance in FastAPI?**
    Logs query metrics.

```python
from fastapi import FastAPI, Request
import time
app = FastAPI()
@app.middleware("http")
async def log_db_performance(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    print(f"DB query took {time.time() - start}s")
    return response
```

Serving ML Models

Basic

55. **How do you serve a machine learning model with FastAPI?**
    Exposes model predictions via API.

```python
from fastapi import FastAPI
import joblib
app = FastAPI()
model = joblib.load("model.pkl")
@app.post("/predict")
async def predict(data: list):
    return {"prediction": model.predict([data]).tolist()}
```

56. **What is Pydantic for model input validation?**
    Ensures valid input data.

```python
from pydantic import BaseModel
class PredictionInput(BaseModel):
    features: list[float]
```

57. **How do you integrate FastAPI with scikit-learn?**
    Serves scikit-learn models.

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
app = FastAPI()
model = joblib.load("sklearn_model.pkl")
class Input(BaseModel):
    features: list[float]
@app.post("/predict")
async def predict(input: Input):
    return {"prediction": model.predict([input.features]).tolist()}
```

58. **How do you serve a PyTorch model with FastAPI?**
    Uses PyTorch for inference.

```python
from fastapi import FastAPI
import torch
app = FastAPI()
model = torch.load("pytorch_model.pth")
model.eval()
@app.post("/predict")
async def predict(data: list):
    input_tensor = torch.tensor(data)
    return {"prediction": model(input_tensor).tolist()}
```

59. **How do you handle large model inputs?**
    Processes inputs in chunks.

```python
from fastapi import FastAPI
app = FastAPI()
@app.post("/predict")
async def predict(data: list):
    return {"chunked": len(data) // 100}
```

60. **How do you visualize model prediction performance?**
    Plots accuracy or latency.

```python
import matplotlib.pyplot as plt
def plot_prediction_performance(metrics):
    plt.plot(metrics["accuracy"])
    plt.savefig("prediction_performance.png")
```

Intermediate

61. **Write a function to serve a TensorFlow model.**
    Uses TensorFlow for predictions.

```python
from fastapi import FastAPI
from pydantic import BaseModel
import tensorflow as tf
app = FastAPI()
model = tf.keras.models.load_model("tf_model")
class Input(BaseModel):
    features: list[float]
@app.post("/predict")
async def predict(input: Input):
    return {"prediction": model.predict([input.features]).tolist()}
```

62. **How do you implement batch predictions in FastAPI?**
    Processes multiple inputs.

```python
from fastapi import FastAPI
import joblib
app = FastAPI()
model = joblib.load("model.pkl")
@app.post("/batch_predict")
async def batch_predict(data: list[list[float]]):
    return {"predictions": model.predict(data).tolist()}
```

63. **Write a function to integrate FastAPI with ONNX models.**
    Uses ONNX runtime for inference.

```python
from fastapi import FastAPI
import onnxruntime as ort
app = FastAPI()
session = ort.InferenceSession("model.onnx")
@app.post("/predict")
async def predict(data: list):
    return {"prediction": session.run(None, {"input": data})[0].tolist()}
```

64. **How do you serve Hugging Face models with FastAPI?**
Uses Transformers for NLP tasks.

```python
from fastapi import FastAPI
from transformers import pipeline
app = FastAPI()
nlp = pipeline("sentiment-analysis")
@app.post("/sentiment")
async def sentiment(text: str):
    return nlp(text)
```

65. **Write a function to cache model predictions.**
Reduces inference time.

```python
from functools import lru_cache
@lru_cache(maxsize=1000)
def cached_predict(data: str):
    model = joblib.load("model.pkl")
    return model.predict([eval(data)]).tolist()
```

66. **How do you handle model versioning in FastAPI?**
Serves different model versions.

```python
from fastapi import FastAPI
import joblib
app = FastAPI()
models = {v: joblib.load(f"model_v{v}.pkl") for v in [1, 2]}
@app.post("/predict/{version}")
async def predict(version: int, data: list):
    return {"prediction": models[version].predict([data]).tolist()}
```

Advanced

67. **Write a function to implement A/B testing for models.**
Compares model performance.

```python
from fastapi import FastAPI
import joblib
app = FastAPI()
models = {"A": joblib.load("model_a.pkl"), "B": joblib.load("model_b.pkl")}
@app.post("/ab_test")
async def ab_test(data: list):
    return {
        "A": models["A"].predict([data]).tolist(),
        "B": models["B"].predict([data]).tolist()
    }
```

68. **How do you implement model monitoring in FastAPI?**
Logs prediction metrics.

```python
from fastapi import FastAPI
import joblib
app = FastAPI()
model = joblib.load("model.pkl")
@app.post("/predict")
async def predict(data: list):
    prediction = model.predict([data]).tolist()
    print(f"Prediction: {prediction}")
    return {"prediction": prediction}
```

69. **Write a function to serve streaming model predictions.**
Streams large outputs.

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
app = FastAPI()
model = joblib.load("model.pkl")
async def stream_predictions(data: list):
    for chunk in data:
        yield str(model.predict([chunk]).tolist()) + "\n"
@app.post("/stream")
async def stream(data: list):
    return StreamingResponse(stream_predictions(data),
media_type="text/plain")
```

70. **How do you optimize model inference in FastAPI?**
Uses batching and async.

```python
from fastapi import FastAPI
import joblib
app = FastAPI()
model = joblib.load("model.pkl")
@app.post("/optimized_predict")
async def optimized_predict(data: list[list[float]]):
    return {"predictions": model.predict(data).tolist()}
```

71. **Write a function to integrate FastAPI with Triton Inference Server.**
Serves models via Triton.

```python
from fastapi import FastAPI
from tritonclient.http import InferenceServerClient
app = FastAPI()
triton_client = InferenceServerClient("localhost:8000")
@app.post("/triton_predict")
async def triton_predict(data: list):
    inputs = triton_client.infer("model", [data])
    return {"prediction": inputs.as_numpy("output").tolist()}
```

72. **How do you implement model explainability in FastAPI?**
Returns feature importance.

```python
from fastapi import FastAPI
import joblib
import shap
app = FastAPI()
model = joblib.load("model.pkl")
explainer = shap.TreeExplainer(model)
@app.post("/explain")
async def explain(data: list):
    shap_values = explainer.shap_values([data])
    return {"shap_values": shap_values.tolist()}
```

Testing and Validation

Basic

73. **How do you write unit tests for FastAPI endpoints?**
Uses TestClient for testing.

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
app = FastAPI()
@app.get("/")
async def root():
    return {"message": "Hello"}
client = TestClient(app)
def test_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello"}
```

74. **What is Pydantic validation testing?**
Tests input validation.

```python
from pydantic import BaseModel
class Item(BaseModel):
    name: str
def test_pydantic_validation():
    try:
        Item(name=123)
        assert False
    except:
        assert True
```

75. **How do you test query parameters in FastAPI?**
Simulates query strings.

```python
from fastapi.testclient import TestClient
from fastapi import FastAPI
app = FastAPI()
@app.get("/items")
async def read_items(q: str):
    return {"q": q}
client = TestClient(app)
def test_query():
    response = client.get("/items?q=test")
    assert response.json() == {"q": "test"}
```

76. **How do you mock dependencies in FastAPI tests?**
Overrides dependencies.

```python
from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient
app = FastAPI()
async def get_db():
    return {"db": "real"}
@app.get("/data")
async def read_data(db=Depends(get_db)):
    return db
client = TestClient(app)
def test_mock_db():
    app.dependency_overrides[get_db] = lambda: {"db": "mock"}
```

```python
response = client.get("/data")
assert response.json() == {"db": "mock"}
```

77. **How do you test error handling in FastAPI?**
    Simulates error conditions.

```python
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
app = FastAPI()
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id < 0:
        raise HTTPException(status_code=400, detail="Invalid ID")
    return {"item_id": item_id}
client = TestClient(app)
def test_error():
    response = client.get("/items/-1")
    assert response.status_code == 400
```

78. **How do you visualize test coverage?**
    Plots coverage metrics.

```python
import matplotlib.pyplot as plt
def plot_test_coverage(coverage):
    plt.bar(["Covered", "Uncovered"], [coverage, 100 - coverage])
    plt.savefig("test_coverage.png")
```

Intermediate

79. **Write a function to test async endpoints.**
    Tests asynchronous routes.

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
app = FastAPI()
@app.get("/async")
async def async_endpoint():
    return {"status": "async"}
client = TestClient(app)
def test_async():
    response = client.get("/async")
    assert response.status_code == 200
    assert response.json() == {"status": "async"}
```

80. **How do you implement integration tests for FastAPI?**
    Tests multiple components.

```python
from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient
app = FastAPI()
def get_db():
    return {"db": "connected"}
@app.get("/data")
async def read_data(db=Depends(get_db)):
    return db
client = TestClient(app)
def test_integration():
    response = client.get("/data")
    assert response.json() == {"db": "connected"}
```

81. **Write a function to test WebSocket endpoints.**
    Simulates WebSocket connections.

```python
from fastapi import FastAPI, WebSocket
from starlette.testclient import TestClient
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    await websocket.send_text("Test")
    await websocket.close()
client = TestClient(app)
def test_websocket():
    with client.websocket_connect("/ws") as ws:
        assert ws.receive_text() == "Test"
```

82. **How do you test middleware in FastAPI?**
    Verifies middleware behavior.

```python
from fastapi import FastAPI, Request
from fastapi.testclient import TestClient
app = FastAPI()
@app.middleware("http")
async def add_header(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Test"] = "Value"
    return response
@app.get("/")
async def root():
```

```python
    return {"message": "Hello"}
client = TestClient(app)
def test_middleware():
    response = client.get("/")
    assert response.headers["X-Test"] == "Value"
```

83. **Write a function to test database interactions.**
    Tests CRUD operations.

```python
from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient
app = FastAPI()
def get_db():
    return Session()
@app.post("/users")
async def create_user(name: str, db=Depends(get_db)):
    user = User(name=name)
    db.add(user)
    db.commit()
    return {"name": name}
client = TestClient(app)
def test_db():
    response = client.post("/users", json={"name": "Test"})
    assert response.json() == {"name": "Test"}
```

84. **How do you implement load testing for FastAPI?**
    Simulates high traffic.

```python
from locust import HttpUser, task
class FastAPIUser(HttpUser):
    host = "http://localhost:8000"
    @task
    def test_endpoint(self):
        self.client.get("/")
```

Advanced

85. **Write a function to implement end-to-end testing.**
    Tests full API workflows.

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
app = FastAPI()
@app.post("/items")
async def create_item(name: str):
    return {"name": name}
@app.get("/items/{name}")
async def read_item(name: str):
    return {"name": name}
client = TestClient(app)
def test_e2e():
    create_response = client.post("/items", json={"name": "Test"})
    read_response = client.get("/items/Test")
    assert create_response.json() == read_response.json()
```

86. **How do you implement fuzz testing for FastAPI?**
    Tests with random inputs.

```python
from fastapi.testclient import TestClient
from hypothesis import given
from hypothesis.strategies import text
app = FastAPI()
@app.get("/echo/{value}")
async def echo(value: str):
    return {"value": value}
client = TestClient(app)
@given(text())
def test_fuzz(value):
    response = client.get(f"/echo/{value}")
    assert response.status_code == 200
```

87. **Write a function to test API versioning.**
    Verifies versioned endpoints.

```python
from fastapi import FastAPI, APIRouter
from fastapi.testclient import TestClient
app = FastAPI()
v1 = APIRouter(prefix="/v1")
@v1.get("/items")
async def read_items_v1():
    return {"version": "v1"}
app.include_router(v1)
client = TestClient(app)
def test_versioning():
    response = client.get("/v1/items")
    assert response.json() == {"version": "v1"}
```

88. **How do you test rate limiting in FastAPI?**
    Simulates excessive requests.

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
from slowapi import Limiter
app = FastAPI()
limiter = Limiter(key_func=lambda: "test")
app.state.limiter = limiter
@app.get("/limited")
@limiter.limit("1/second")
async def limited():
    return {"status": "OK"}
client = TestClient(app)
def test_rate_limit():
    response1 = client.get("/limited")
    response2 = client.get("/limited")
    assert response2.status_code == 429
```

89. **Write a function to test dependency injection.**
    Verifies dependency behavior.

```python
from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient
app = FastAPI()
async def get_token():
    return "test_token"
@app.get("/secure")
async def secure_endpoint(token=Depends(get_token)):
    return {"token": token}
client = TestClient(app)
def test_dependency():
    response = client.get("/secure")
    assert response.json() == {"token": "test_token"}
```

90. **How do you implement performance testing for FastAPI?**
    Measures throughput and latency.

```python
import time
from fastapi.testclient import TestClient
app = FastAPI()
@app.get("/")
async def root():
    return {"message": "Hello"}
client = TestClient(app)
def test_performance():
    start = time.time()
    for _ in range(100):
        client.get("/")
    print(f"Average latency: {(time.time() - start) / 100}s")
```

Deployment and Scalability

Basic

91. **How do you deploy a FastAPI application with Docker?**
    Containerizes the app.

```python
# Dockerfile
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install fastapi uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
```

92. **How do you configure Uvicorn for production?**
    Sets workers and logging.

```python
# Terminal command
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4 --log-level info
```

93. **What is environment configuration in FastAPI?**
    Uses environment variables.

```python
import os
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def root():
    return {"env": os.getenv("ENV", "dev")}
```

94. **How do you enable CORS in FastAPI?**
    Allows cross-origin requests.

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
```

```python
    allow_methods=["*"],
    allow_headers=["*"],
)
```

95. **How do you deploy FastAPI to AWS Lambda?**
    Uses Mangum for ASGI.

```python
from fastapi import FastAPI
from mangum import Mangum
app = FastAPI()
@app.get("/")
async def root():
    return {"message": "Hello"}
handler = Mangum(app)
```

96. **How do you visualize deployment metrics?**
    Plots latency and throughput.

```python
import matplotlib.pyplot as plt
def plot_deployment_metrics(metrics):
    plt.plot(metrics["latency"], label="Latency")
    plt.plot(metrics["throughput"], label="Throughput")
    plt.legend()
    plt.savefig("deployment_metrics.png")
```

Intermediate

97. **Write a function to deploy FastAPI with Kubernetes.**
    Defines Kubernetes manifests.

```python
from kubernetes import client, config
def deploy_fastapi():
    config.load_kube_config()
    v1 = client.CoreV1Api()
    service = client.V1Service(
        metadata=client.V1ObjectMeta(name="fastapi-service"),
        spec=client.V1ServiceSpec(
            selector={"app": "fastapi"},
            ports=[client.V1ServicePort(port=80)]
        )
    )
    v1.create_namespaced_service(namespace="default", body=service)
```

98. **How do you implement load balancing in FastAPI?**
    Uses nginx or Kubernetes.

```python
# nginx.conf snippet
upstream fastapi {
    server 127.0.0.1:8000;
    server 127.0.0.1:8001;
}
```

99. **Write a function to monitor FastAPI performance.**
    Logs request metrics.

```python
from fastapi import FastAPI, Request
import time
app = FastAPI()
@app.middleware("http")
async def monitor_performance(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    print(f"Request took {time.time() - start}s")
    return response
```

00. **How do you implement auto-scaling for FastAPI?**
    Uses Kubernetes HPA.

```python
from kubernetes import client, config
def create_hpa():
    config.load_kube_config()
    v1 = client.AutoscalingV1Api()
    hpa = client.V1HorizontalPodAutoscaler(
        metadata=client.V1ObjectMeta(name="fastapi-hpa"),
        spec=client.V1HorizontalPodAutoscalerSpec(
            scale_target_ref=client.V1CrossVersionObjectReference(
                kind="Deployment", name="fastapi", api_version="apps/v1"
            ),
            min_replicas=1,
            max_replicas=10,
            target_cpu_utilization_percentage=80
        )
    )
    v1.create_namespaced_horizontal_pod_autoscaler(namespace="default",
body=hpa)
```

01. **Write a function to handle graceful shutdown.**
Ensures clean termination.

```python
from fastapi import FastAPI
import asyncio
app = FastAPI()
@app.on_event("shutdown")
async def shutdown_event():
    print("Shutting down...")
    await asyncio.sleep(1)
```

02. **How do you implement health checks in FastAPI?**
Exposes health endpoints.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/health")
async def health_check():
    return {"status": "healthy"}
```

Advanced

03. **Write a function to implement blue-green deployment.**
Switches between app versions.

```python
from kubernetes import client, config
def blue_green_deploy(version: str):
    config.load_kube_config()
    v1 = client.CoreV1Api()
    service = client.V1Service(
        metadata=client.V1ObjectMeta(name="fastapi-service"),
        spec=client.V1ServiceSpec(
            selector={"app": f"fastapi-{version}"},
            ports=[client.V1ServicePort(port=80)]
        )
    )
    v1.replace_namespaced_service(name="fastapi-service",
namespace="default", body=service)
```

04. **How do you integrate FastAPI with Prometheus?**
Exposes metrics for monitoring.

```python
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
app = FastAPI()
Instrumentator().instrument(app).expose(app)
@app.get("/")
async def root():
    return {"message": "Hello"}
```

05. **Write a function to implement canary releases.**
Tests new versions gradually.

```python
from kubernetes import client, config
def canary_release():
    config.load_kube_config()
    v1 = client.AppsV1Api()
    deployment = client.V1Deployment(
        metadata=client.V1ObjectMeta(name="fastapi-canary"),
        spec=client.V1DeploymentSpec(
            replicas=1,
            selector=client.V1LabelSelector(match_labels={"app": "fastapi-
canary"}),
            template=client.V1PodTemplateSpec(
                metadata=client.V1ObjectMeta(labels={"app": "fastapi-
canary"}),
                spec=client.V1PodSpec(containers=
[client.V1Container(name="fastapi", image="fastapi:latest")])
            )
        )
    )
    v1.create_namespaced_deployment(namespace="default", body=deployment)
```

06. **How do you optimize FastAPI for low-latency?**
Uses async and caching.

```python
from fastapi import FastAPI
import redis.asyncio as redis
app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
@app.get("/cached")
async def cached_endpoint(key: str):
    value = await r.get(key)
    if value:
        return {"value": value.decode()}
    return {"value": "computed"}
```

07. **Write a function to implement distributed tracing.**
Tracks request flow.

```python
from fastapi import FastAPI
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
app = FastAPI()
FastAPIInstrumentor.instrument_app(app)
@app.get("/")
async def root():
    return {"message": "Traced"}
```

08. **How do you handle failover in FastAPI deployments?**
Uses multiple replicas.

```python
from kubernetes import client, config
def ensure_replicas():
    config.load_kube_config()
    v1 = client.AppsV1Api()
    deployment = client.V1Deployment(
        metadata=client.V1ObjectMeta(name="fastapi"),
        spec=client.V1DeploymentSpec(replicas=3,
selector=client.V1LabelSelector(match_labels={"app": "fastapi"}))
    )
    v1.create_namespaced_deployment(namespace="default", body=deployment)
```

Security

Basic

09. **How do you implement authentication in FastAPI?**
Uses OAuth2 or JWT.

```python
from fastapi import FastAPI, Depends
from fastapi.security import OAuth2PasswordBearer
app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
@app.get("/secure")
async def secure_endpoint(token=Depends(oauth2_scheme)):
    return {"token": token}
```

10. **What is input validation in FastAPI?**
Uses Pydantic to validate data.

```python
from pydantic import BaseModel
class User(BaseModel):
    username: str
    age: int
```

11. **How do you enable HTTPS in FastAPI?**
Configures SSL with Uvicorn.

```python
# Terminal command
uvicorn main:app --ssl-keyfile=key.pem --ssl-certfile=cert.pem
```

12. **How do you prevent SQL injection in FastAPI?**
Uses parameterized queries.

```python
from sqlalchemy.orm import Session
def get_user(db: Session, user_id: int):
    return db.query(User).filter(User.id == user_id).first()
```

13. **How do you secure sensitive environment variables?**
Uses python-dotenv.

```python
from dotenv import load_dotenv
import os
load_dotenv()
API_KEY = os.getenv("API_KEY")
```

14. **How do you visualize security metrics?**
Plots authentication failures.

```python
import matplotlib.pyplot as plt
def plot_security_metrics(failures):
    plt.plot(failures)
    plt.savefig("security_metrics.png")
```

Intermediate

15. **Write a function to implement JWT authentication.**
Verifies JWT tokens.

```python
from fastapi import FastAPI, Depends, HTTPException
from jose import JWTError, jwt
app = FastAPI()
```

```python
SECRET_KEY = "secret"
async def verify_token(token: str =
Depends(OAuth2PasswordBearer(tokenUrl="token"))):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return payload
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
@app.get("/secure")
async def secure_endpoint(payload=Depends(verify_token)):
    return {"user": payload}
```

16. **How do you implement role-based access control?**
    Checks user roles.

```python
from fastapi import FastAPI, Depends, HTTPException
app = FastAPI()
async def check_role(token: str =
Depends(OAuth2PasswordBearer(tokenUrl="token"))):
    if token != "admin":
        raise HTTPException(status_code=403, detail="Forbidden")
    return token
@app.get("/admin")
async def admin_endpoint(token=Depends(check_role)):
    return {"access": "admin"}
```

17. **Write a function to prevent CSRF attacks.**
    Uses CSRF tokens.

```python
from fastapi import FastAPI, Request, HTTPException
app = FastAPI()
@app.post("/action")
async def action(request: Request):
    token = request.headers.get("X-CSRF-Token")
    if not token or token != "valid_token":
        raise HTTPException(status_code=403, detail="Invalid CSRF token")
    return {"status": "OK"}
```

18. **How do you implement rate limiting for security?**
    Limits request frequency.

```python
from fastapi import FastAPI
from slowapi import Limiter
app = FastAPI()
limiter = Limiter(key_func=lambda: "user")
app.state.limiter = limiter
@app.get("/secure")
@limiter.limit("10/minute")
async def secure_endpoint():
    return {"status": "OK"}
```

19. **Write a function to log security events.**
    Tracks unauthorized access.

```python
import logging
from fastapi import FastAPI, HTTPException
app = FastAPI()
logging.basicConfig(filename="security.log", level=logging.INFO)
@app.get("/secure")
async def secure_endpoint():
    try:
        raise HTTPException(status_code=401, detail="Unauthorized")
    except HTTPException as e:
        logging.error(f"Security event: {e.detail}")
        raise
```

20. **How do you secure WebSocket endpoints?**
    Validates tokens.

```python
from fastapi import FastAPI, WebSocket, HTTPException
app = FastAPI()
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    token = await websocket.receive_text()
    if token != "valid_token":
        await websocket.close(code=1008)
        return
    await websocket.send_text("Authenticated")
```

Advanced

21. **Write a function to implement API key authentication.**
    Verifies API keys.

```python
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import APIKeyHeader
app = FastAPI()
api_key_header = APIKeyHeader(name="X-API-Key")
```

```python
async def verify_api_key(api_key: str = Depends(api_key_header)):
    if api_key != "secret_key":
        raise HTTPException(status_code=401, detail="Invalid API key")
    return api_key
@app.get("/secure")
async def secure_endpoint(api_key=Depends(verify_api_key)):
    return {"status": "Authenticated"}
```

22. **How do you implement encryption for sensitive data?**
    Uses Fernet for encryption.

```python
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
def encrypt_data(data: str):
    return f.encrypt(data.encode()).decode()
```

23. **Write a function to detect injection attacks.**
    Sanitizes inputs.

```python
from fastapi import FastAPI, HTTPException
app = FastAPI()
def sanitize_input(input: str):
    if any(c in input for c in [";", "--", "<script>"]):
        raise HTTPException(status_code=400, detail="Invalid input")
    return input
@app.get("/search")
async def search(q: str):
    return {"query": sanitize_input(q)}
```

24. **How do you implement audit logging for security?**
    Logs all requests.

```python
from fastapi import FastAPI, Request
import logging
app = FastAPI()
logging.basicConfig(filename="audit.log", level=logging.INFO)
@app.middleware("http")
async def audit_log(request: Request, call_next):
    logging.info(f"Request: {request.method} {request.url}")
    return await call_next(request)
```

25. **Write a function to implement secure file uploads.**
    Validates file types.

```python
from fastapi import FastAPI, File, UploadFile, HTTPException
app = FastAPI()
@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    if file.content_type not in ["image/png", "image/jpeg"]:
        raise HTTPException(status_code=400, detail="Invalid file type")
    return {"filename": file.filename}
```

26. **How do you integrate FastAPI with OAuth2 providers?**
    Uses Authlib for OAuth2.

```python
from fastapi import FastAPI
from authlib.integrations.starlette_client import OAuth
app = FastAPI()
oauth = OAuth()
oauth.register(
    name="google",
    client_id="client_id",
    client_secret="client_secret",
    authorize_url="https://accounts.google.com/o/oauth2/auth",
    token_url="https://accounts.google.com/o/oauth2/token"
)
@app.get("/login")
async def login():
    return {"url": oauth.google.create_authorize_url()}
```

Integration with Other Tools

Basic

27. **How do you integrate FastAPI with Celery?**
    Runs async tasks.

```python
from fastapi import FastAPI
from celery import Celery
app = FastAPI()
celery_app = Celery("tasks", broker="redis://localhost:6379")
@celery_app.task
def process_task(data: str):
    return f"Processed: {data}"
@app.post("/task")
async def create_task(data: str):
```

```
    process_task.delay(data)
    return {"status": "Task queued"}
```

28. **How do you integrate FastAPI with Redis?**
    Uses Redis for caching.

```python
import redis.asyncio as redis
from fastapi import FastAPI
app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
@app.get("/cache/{key}")
async def read_cache(key: str):
    value = await r.get(key)
    return {"value": value.decode() if value else None}
```

29. **How do you integrate FastAPI with Kafka?**
    Streams data via Kafka.

```python
from fastapi import FastAPI
from aiokafka import AIOKafkaProducer
app = FastAPI()
producer = AIOKafkaProducer(bootstrap_servers="localhost:9092")
@app.on_event("startup")
async def startup():
    await producer.start()
@app.post("/publish")
async def publish(message: str):
    await producer.send_and_wait("topic", message.encode())
    return {"status": "Published"}
```

30. **How do you integrate FastAPI with SQLAlchemy?**
    Uses ORM for database access.

```python
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
app = FastAPI()
def get_db():
    db = Session()
    try:
        yield db
    finally:
        db.close()
@app.get("/users")
```

```
async def read_users(db=Depends(get_db)):
    return db.query(User).all()
```

31. **How do you integrate FastAPI with Prometheus?**
    Exposes metrics.

```python
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
app = FastAPI()
Instrumentator().instrument(app).expose(app)
@app.get("/")
async def root():
    return {"message": "Monitored"}
```

32. **How do you visualize integration metrics?**
    Plots task or cache metrics.

```python
import matplotlib.pyplot as plt
def plot_integration_metrics(metrics):
    plt.plot(metrics["cache_hits"], label="Cache Hits")
    plt.plot(metrics["task_completion"], label="Tasks")
    plt.legend()
    plt.savefig("integration_metrics.png")
```

Intermediate

33. **Write a function to integrate FastAPI with Airflow.**
    Triggers Airflow DAGs.

```python
from fastapi import FastAPI
from airflow.api.client.local_client import Client
app = FastAPI()
client = Client()
@app.post("/trigger_dag")
async def trigger_dag(dag_id: str):
    client.trigger_dag(dag_id=dag_id)
    return {"status": f"DAG {dag_id} triggered"}
```

34. **How do you integrate FastAPI with Elasticsearch?**
    Queries Elasticsearch indices.

```python
from fastapi import FastAPI
from elasticsearch import AsyncElasticsearch
```

```python
app = FastAPI()
es = AsyncElasticsearch(["http://localhost:9200"])
@app.get("/search")
async def search(q: str):
    result = await es.search(index="my_index", query={"match": {"text":
q}})
    return result["hits"]["hits"]
```

35. **Write a function to integrate FastAPI with RabbitMQ.**
    Publishes messages.

```python
from fastapi import FastAPI
import aio_pika
app = FastAPI()
@app.on_event("startup")
async def connect_rabbit():
    global connection
    connection = await
aio_pika.connect_robust("amqp://guest:guest@localhost/")
@app.post("/publish")
async def publish(message: str):
    async with connection:
        channel = await connection.channel()
        await channel.default_exchange.publish(
            aio_pika.Message(message.encode()), routing_key="queue"
        )
    return {"status": "Published"}
```

36. **How do you integrate FastAPI with GraphQL?**
    Uses Ariadne for GraphQL APIs.

```python
from fastapi import FastAPI
from ariadne import QueryType, gql, make_executable_schema
from ariadne.asgi import GraphQL
app = FastAPI()
type_defs = gql("""
    type Query {
        hello: String!
    }
""")
query = QueryType()
@query.field("hello")
def resolve_hello(_, info):
    return "Hello, GraphQL!"
schema = make_executable_schema(type_defs, query)
app.mount("/graphql", GraphQL(schema))
```

37. **Write a function to integrate FastAPI with MLflow.**
    Tracks ML experiments.

```python
from fastapi import FastAPI
import mlflow
app = FastAPI()
@app.post("/log_experiment")
async def log_experiment(params: dict, metrics: dict):
    with mlflow.start_run():
        mlflow.log_params(params)
        mlflow.log_metrics(metrics)
    return {"status": "Logged"}
```

38. **How do you integrate FastAPI with LangChain?**
    Serves LLM prompts.

```python
from fastapi import FastAPI
from langchain.prompts import PromptTemplate
app = FastAPI()
prompt = PromptTemplate.from_template("Task: {task}")
@app.post("/prompt")
async def create_prompt(task: str):
    return {"prompt": prompt.format(task=task)}
```

Advanced

39. **Write a function to integrate FastAPI with Apache Spark.**
    Processes big data.

```python
from fastapi import FastAPI
from pyspark.sql import SparkSession
app = FastAPI()
spark = SparkSession.builder.appName("FastAPI-Spark").getOrCreate()
@app.post("/process")
async def process_data(data: list):
    df = spark.createDataFrame(data)
    result = df.groupBy("category").count().collect()
    return {"result": result}
```

40. **How do you integrate FastAPI with Kubernetes Ingress?**
Routes external traffic.

```python
from kubernetes import client, config
def create_ingress():
    config.load_kube_config()
    v1 = client.NetworkingV1Api()
    ingress = client.V1Ingress(
        metadata=client.V1ObjectMeta(name="fastapi-ingress"),
        spec=client.V1IngressSpec(
            rules=[client.V1IngressRule(
                host="api.example.com",
                http=client.V1HTTPIngressRuleValue(
                    paths=[client.V1HTTPIngressPath(
                        path="/", path_type="Prefix",
                        backend=client.V1IngressBackend(
                            service=client.V1IngressServiceBackend(
                                name="fastapi-service",
port=client.V1ServiceBackendPort(number=80)
                            )
                        )
                    )]
                )
            )]
        )
    )
    v1.create_namespaced_ingress(namespace="default", body=ingress)
```

41. **Write a function to integrate FastAPI with AWS SQS.**
Processes queue messages.

```python
from fastapi import FastAPI
import boto3
app = FastAPI()
sqs = boto3.client("sqs", region_name="us-east-1")
@app.post("/queue")
async def send_message(message: str):
    sqs.send_message(QueueUrl="queue_url", MessageBody=message)
    return {"status": "Message sent"}
```

42. **How do you integrate FastAPI with TensorFlow Serving?**
Serves TensorFlow models.

```python
from fastapi import FastAPI
import requests
app = FastAPI()
@app.post("/tf_predict")
async def tf_predict(data: list):
    response =
requests.post("http://localhost:8501/v1/models/model:predict", json=
{"instances": [data]})
    return response.json()
```

43. **Write a function to integrate FastAPI with Dask.**
Handles distributed computing.

```python
from fastapi import FastAPI
from dask.distributed import Client
app = FastAPI()
dask_client = Client("tcp://scheduler:8786")
@app.post("/compute")
async def compute(data: list):
    result = dask_client.submit(sum, data).result()
    return {"result": result}
```

44. **How do you integrate FastAPI with OpenTelemetry?**
Traces API requests.

```python
from fastapi import FastAPI
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
app = FastAPI()
FastAPIInstrumentor.instrument_app(app)
@app.get("/")
async def root():
    return {"message": "Traced"}
```

Performance Optimization

Basic

45. **How do you optimize FastAPI response times?**
Uses async endpoints.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/fast")
```

```python
async def fast_endpoint():
    return {"status": "OK"}
```

46. **What is connection pooling in FastAPI?**
    Reuses database connections.

```python
engine = create_engine("sqlite:///example.db", pool_size=5,
max_overflow=10)
```

47. **How do you cache API responses?**
    Uses Redis for caching.

```python
import redis.asyncio as redis
from fastapi import FastAPI
app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
@app.get("/cached")
async def cached_endpoint():
    value = await r.get("key")
    if value:
        return {"value": value.decode()}
    return {"value": "computed"}
```

48. **How do you reduce memory usage in FastAPI?**
    Clears unused objects.

```python
import gc
from fastapi import FastAPI
app = FastAPI()
@app.get("/low_memory")
async def low_memory_endpoint():
    gc.collect()
    return {"status": "OK"}
```

49. **How do you profile FastAPI performance?**
    Uses cProfile for analysis.

```python
import cProfile
from fastapi.testclient import TestClient
app = FastAPI()
client = TestClient(app)
cProfile.run('client.get("/")', "fastapi_profile.prof")
```

50. **How do you visualize performance metrics?**
    Plots latency and memory usage.

```python
import matplotlib.pyplot as plt
def plot_performance_metrics(metrics):
    plt.plot(metrics["latency"], label="Latency")
    plt.plot(metrics["memory"], label="Memory")
    plt.legend()
    plt.savefig("performance_metrics.png")
```

Intermediate

51. **Write a function to implement response compression.**
    Reduces response size.

```python
from fastapi import FastAPI
from fastapi.middleware.gzip import GZipMiddleware
app = FastAPI()
app.add_middleware(GZipMiddleware, minimum_size=1000)
@app.get("/large")
async def large_response():
    return {"data": "x" * 10000}
```

52. **How do you implement batch processing in FastAPI?**
    Processes multiple requests.

```python
from fastapi import FastAPI
app = FastAPI()
@app.post("/batch")
async def batch_process(data: list[dict]):
    return [{"processed": item} for item in data]
```

53. **Write a function to optimize database queries.**
    Uses indexing.

```python
from sqlalchemy import Index
def optimize_db():
    Index("idx_user_name", User.name).create(bind=engine)
```

54. **How do you implement async task offloading?**
    Uses Celery for tasks.

```python
from fastapi import FastAPI
from celery import Celery
app = FastAPI()
celery_app = Celery("tasks", broker="redis://localhost:6379")
@celery_app.task
def heavy_task(data: str):
    return f"Processed: {data}"
@app.post("/offload")
async def offload_task(data: str):
    heavy_task.delay(data)
    return {"status": "Task offloaded"}
```

55. **Write a function to monitor memory usage.**
    Tracks memory consumption.

```python
import psutil
from fastapi import FastAPI
app = FastAPI()
@app.get("/memory")
async def memory_usage():
    return {"memory": psutil.Process().memory_info().rss / 1024 ** 2}
```

56. **How do you implement connection timeouts?**
    Limits request duration.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/timeout")
async def timeout_endpoint():
    import asyncio
    await asyncio.sleep(1)
    return {"status": "OK"}
# Uvicorn config: --timeout-keep-alive 5
```

Advanced

57. **Write a function to implement request batching.**
    Groups requests for efficiency.

```python
from fastapi import FastAPI
app = FastAPI()
async def batch_process(data: list):
    return [f"Processed: {item}" for item in data]
@app.post("/batch")
async def batch_endpoint(data: list[str]):
    return await batch_process(data)
```

58. **How do you optimize FastAPI for high-throughput?**
    Uses multiple workers and async.

```python
# Terminal command
uvicorn main:app --workers 8 --host 0.0.0.0 --port 8000
```

59. **Write a function to implement distributed caching.**
    Uses Redis Cluster.

```python
from fastapi import FastAPI
from redis.asyncio import Redis
app = FastAPI()
r = Redis.from_url("redis://localhost:6379")
@app.get("/dist_cache/{key}")
async def distributed_cache(key: str):
    value = await r.get(key)
    return {"value": value.decode() if value else None}
```

60. **How do you implement request deduplication?**
    Avoids duplicate processing.

```python
from fastapi import FastAPI
import redis.asyncio as redis
app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
@app.post("/dedup")
async def dedup_request(data: str):
    if await r.setnx(data, 1):
        return {"status": "Processed"}
    return {"status": "Duplicate"}
```

61. **Write a function to implement circuit breakers.**
    Prevents cascading failures.

```python
from pybreaker import CircuitBreaker
from fastapi import FastAPI
app = FastAPI()
breaker = CircuitBreaker(fail_max=3, reset_timeout=60)
@app.get("/reliable")
@breaker
```

```python
async def reliable_endpoint():
    return {"status": "OK"}
```

62. **How do you implement performance benchmarking?**
    Measures endpoint performance.

```python
import time
from fastapi.testclient import TestClient
app = FastAPI()
client = TestClient(app)
def benchmark_endpoint():
    start = time.time()
    for _ in range(1000):
        client.get("/")
    return {"avg_latency": (time.time() - start) / 1000}
```

Monitoring and Logging

Basic

63. **How do you implement logging in FastAPI?**
    Uses Python logging.

```python
import logging
from fastapi import FastAPI
app = FastAPI()
logging.basicConfig(filename="app.log", level=logging.INFO)
@app.get("/")
async def root():
    logging.info("Root endpoint accessed")
    return {"message": "Hello"}
```

64. **How do you expose metrics in FastAPI?**
    Uses Prometheus.

```python
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
app = FastAPI()
Instrumentator().instrument(app).expose(app)
@app.get("/")
async def root():
    return {"message": "Monitored"}
```

65. **How do you implement health checks?**
    Monitors app status.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/health")
async def health_check():
    return {"status": "healthy"}
```

66. **How do you log request details?**
    Captures request metadata.

```python
from fastapi import FastAPI, Request
import logging
app = FastAPI()
logging.basicConfig(filename="requests.log", level=logging.INFO)
@app.middleware("http")
async def log_requests(request: Request, call_next):
    logging.info(f"{request.method} {request.url}")
    return await call_next(request)
```

67. **How do you monitor API uptime?**
    Tracks availability.

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/uptime")
async def uptime():
    return {"status": "up"}
```

68. **How do you visualize monitoring metrics?**
    Plots request counts.

```python
import matplotlib.pyplot as plt
def plot_monitoring_metrics(metrics):
    plt.plot(metrics["requests"], label="Requests")
    plt.legend()
    plt.savefig("monitoring_metrics.png")
```

Intermediate

69. **Write a function to implement distributed logging.**
    Sends logs to a central server.

```python
import logging.handlers
from fastapi import FastAPI
app = FastAPI()
handler = logging.handlers.SocketHandler("log-server", 9090)
logging.getLogger().addHandler(handler)
@app.get("/")
async def root():
    logging.info("Distributed log")
    return {"message": "Logged"}
```

70. **How do you integrate FastAPI with Grafana?**
    Visualizes Prometheus metrics.

```python
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
app = FastAPI()
Instrumentator().instrument(app).expose(app)
@app.get("/")
async def root():
    return {"message": "Monitored"}
# Configure Grafana to scrape /metrics
```